HÉLIOS

OBSERVATOIRE DE LA MÉTÉO SPATIALE

GUIDE EN DIX ÉTAPES · AZURE DATABRICKS · LAKEFLOW · UNITY CATALOG

Du Soleil à Power BI et Fabric Apps : un lakehouse de production

Vous allez construire, sur vos propres données publiques, la chaîne complète d'un data engineer Databricks : ingestion d'API, zone d'atterrissage, médaillon bronze, silver et gold, historisation, contrôles qualité, orchestration, gouvernance et coût mesuré, jusqu'aux données prêtes pour Power BI et Fabric Apps. Ce guide explique **quoi faire et pourquoi** ; le code complet vit dans le dépôt GitHub.

94 ans
d'indice Kp, de 1932 à aujourd'hui

3 sondes
qui mesurent le vent solaire à 1,5 million de km

19 tables
Delta, gouvernées par Unity Catalog

1 job
quotidien, contrôlé, qui alerte s'il échoue

Comment lire ce guide

`chemin` Le fichier exact à ouvrir dans le dépôt.

Vérifier — Le résultat attendu avant de passer à la suite.

Œil du gourou — Le choix d'architecture et sa raison.

`valeur` À saisir tel quel, caractère pour caractère.

Règle d'or : on ne passe à l'étape suivante que lorsque la vérification de l'étape en cours est verte.

Le dépôt GitHub · `helios-databricks`

```
00_unity_catalog/        setup_unity_catalog.sql
01_notebooks/            00_explorer.py
                         01_collecteur.py
                         02_controle.py
02_pipeline/
  transformations/       01_bronze.py
                         02_silver_noaa.py
                         03_silver_gfz.py
                         04_silver_donki.py
                         05_gold.sql
03_verifications/        verifications.sql
04_gouvernance/          gouvernance.sql · qualite.sql
                         cout.sql
05_restitution/          tableau_de_bord.sql · genie.md
docs/                    architecture.svg
```

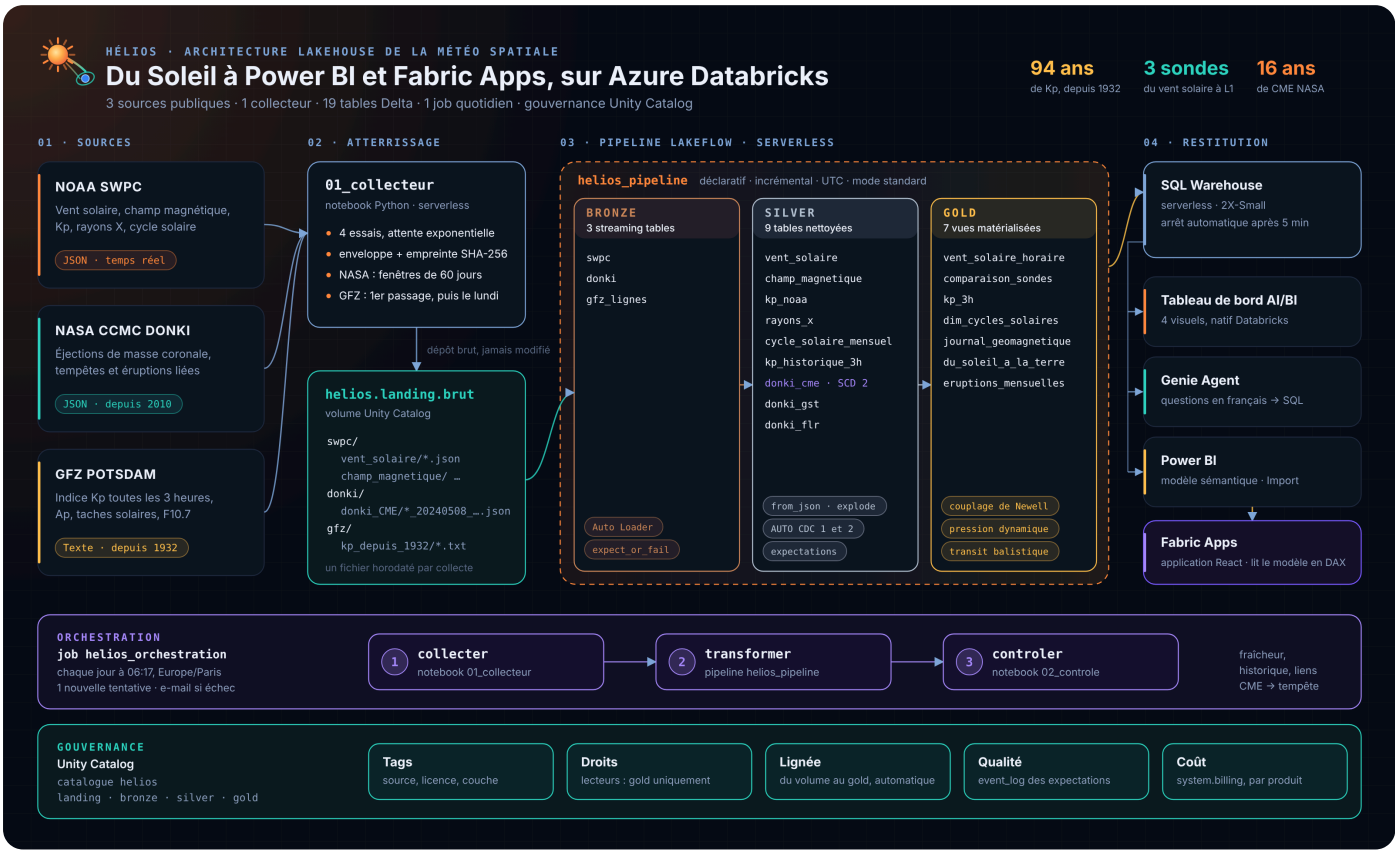
Abdoul Hamid Diallo · Data & AI Engineer · octobre 2026

Données : NOAA SWPC · NASA CCMC DONKI · GFZ Potsdam (CC BY 4.0)

VUE D'ENSEMBLE

L'architecture, en une image

Chaque donnée suit le même chemin, de gauche à droite. Gardez ce schéma sous les yeux : chaque étape du guide en construit une boîte.



ÉTAPES 0 À 3
Atterrir
Le collecteur dépose chaque réponse d'API, intacte et horodatée, dans un volume.

ÉTAPE 4
Bronze
Auto Loader lit seulement les fichiers nouveaux : une ligne par fichier.

ÉTAPE 5
Silver
Décoder, contrôler, dédoublonner (AUTO CDC) et historiser (SCD 2).

ÉTAPE 6
Gold
Sept vues matérialisées qui répondent chacune à une question.

ÉTAPES 7 À 10
Exploiter
Orchestrer, gouverner, mesurer le coût, puis restituer.

L'ŒIL DU GOUROU · DEUX TEMPS, PAS UN

Le collecteur n'écrit jamais directement dans une table. Il dépose d'abord des fichiers, que le pipeline charge ensuite. On garde ainsi la preuve brute de ce que la NOAA ou la NASA a répondu ce jour-là, on peut tout recharger si une règle change, et une panne d'un côté ne bloque pas l'autre.

LES QUATRE PIÈGES DES SOURCES

NOAA	3 sondes pour la même minute ; fenêtres de 24 h qui se recouvrent
NASA	JSON imbriqué ; fiches révisées après coup (`versionId`)
NASA	API limitée à 60 jours par appel
GFZ	texte à colonnes, 8 Kp par ligne, `-1` pour « manquant »

Chaque piège a sa réponse dans le code : c'est ce qui fait un vrai projet.

PHASE 1 · ATTERRIR

00 Préparer le terrain

Un espace prêt, un entrepôt économe, tout le code importé en une fois.

10 min · coût : nul

1. **Espace Azure Databricks Premium**, avec Unity Catalog et **Serverless** (activés par défaut sur les espaces récents).
2. **SQL Warehouses** : un seul entrepôt, taille **2X-Small**, **Auto stop** à 5 min.
3. **Importer le code du projet** (dépôt public, aucun compte GitHub requis) : **Workspace**, votre dossier personnel, **Create**, **Git folder**. URL : `https://github.com/abdoulhamiddiallo/helios-databricks`, puis **Create Git folder**. Tout le code arrive d'un coup, sans copier-coller.

PHASE 1 · ATTERRIR

01 Ranger avant de charger : Unity Catalog

Un catalogue pour le projet, un schéma par couche, un volume pour les fichiers bruts.

10 min · coût : nul (définitions) · Catalog · SQL Editor

1. **Catalog**, **Create**, **Create a catalog**. Nom `helios`, type **Standard**, **Use default storage** coché, **Create**. Si une fenêtre propose les droits, **Configure later**.
2. **SQL Editor**, choisissez votre entrepôt, ouvrez `00_unity_catalog/setup_unity_catalog.sql` (copiez son contenu) et cliquez **Run all**.

Le script crée quatre schémas, un par couche (`landing` pour l'atterrissage, puis `bronze`, `silver`, `gold`), et le volume `helios.landing.brut`, dont l'adresse `/Volumes/helios/landing/brut` servira au collecteur.

VÉRIFIER

La dernière requête du script, `SHOW SCHEMAS IN helios`, renvoie **bronze, gold, landing, silver** (plus `information_schema`).

L'ŒIL DU GOUROU

Un catalogue par projet : droits, tags, lignée et suppression se gèrent d'un bloc. Le script ajoute aussi commentaires et tags à chaque objet.

PHASE 1 · ATTERRIR

02 Regarder les sources avant de les intégrer

Un data engineer ne charge jamais une source qu'il n'a pas vue. Ce notebook n'écrit rien.

10 min · Serverless

1. Ouvrez `01_notebooks/00_explorer.py` dans votre Git folder. En haut à droite, **Connect**, **Serverless**.
2. Exécutez les quatre cellules une à une avec **Maj + Entrée**.

VÉRIFIER

1 · HTTP 200 et 3 000 à 4 000 mesures. **2** · la même minute, une fois par sonde. **3** · la tempête du **10 mai 2024**, Kp 9, reliée à ses CME. **4** · plus de 34 600 jours depuis 1932.

L'ŒIL DU GOUROU

Chaque piège vu ici (trois sondes, fiches révisées, texte à colonnes) aura sa réponse précise dans le pipeline. Si la cellule 1 renvoie `ConnectionError`, voir le dépannage, page 10.

PHASE 1 · ATTERRIR

03

Faire entrer les données dans Databricks

Le collecteur télécharge les trois sources et dépose chaque réponse, telle quelle, dans le volume. À la fin : des fichiers, pas encore de tables.

15 min dont 10 d'attente · Serverless

Ouvrez `01_notebooks/01_collecteur.py`. Ses sept cellules forment un petit logiciel d'ingestion :

CELLULE	CE QU'ELLE FAIT ET LE DÉTAIL QUI COMPTE
1 Widgets	**Trois champs en haut du notebook** : le même code sert à la reprise historique et au passage quotidien, sans modification
2 Constantes	**Adresses des sources, horodatage** : `HORODATAGE` rend chaque nom de fichier unique : rien n'est jamais écrasé
3 `telecharger`	**Appel HTTP robuste** : 4 essais avec attente exponentielle (10, 20, 40 s) sur erreur réseau ou 5xx ; arrêt immédiat sur 4xx, avec le message de la source
4 `deposer_enveloppe`	**Écrit dans le volume** : la réponse est emballée avec source, URL, date de collecte et empreinte **SHA-256** : la preuve que rien n'a changé
5 NOAA	**Cinq fichiers** : vent solaire, champ magnétique, Kp, rayons X, cycle solaire
6 NASA DONKI	**CME, tempêtes, éruptions** : l'API refuse plus de 60 jours : la période est découpée en fenêtres, avec une pause d'une seconde par politesse
7 GFZ et bilan	**Fichier 1932, tableau récapitulatif** : 5,5 Mo téléchargés au premier passage puis le lundi seulement ; déposé en texte brut

Lancer la reprise historique (une seule fois)

1. **Connect**, **Serverless**, puis **Run all** une première fois pour faire apparaître les widgets (ou exécutez la cellule 1).
2. Widget **donki_depuis** : `2010-01-01`. Widget **gfz** : `oui`. Puis **Run all**. Comptez 8 à 12 minutes : environ 309 appels à la NASA.
3. Une fois terminé, **videz** donki_depuis et remettez gfz sur `auto` : c'est le réglage quotidien du job.

VÉRIFIER

Cellule 7 : environ **315 fichiers** (5 swpc, ~309 donki, 1 gfz). **Catalog** → helios → landing → **Volumes** → brut : dossiers `donki`, `gfz`, `swpc`. Bloc « Étape 3 » de `verifications.sql` : `read_files` les lit sans créer de table.

L'ŒIL DU GOUROU · L'ENVELOPPE

Toutes les API reçoivent le même schéma de cinq colonnes (`source, url, fetched_at, sha256, payload`) : une seule fonction de lecture en bronze, et chaque ligne reste traçable jusqu'à son appel.

LES NEUF MOTS À CONNAÎTRE AVANT LE PIPELINE

Volume
Un dossier de fichiers bruts gouverné par Unity Catalog.

Serverless
La machine est fournie à la demande, puis éteinte.

Table Delta
Une table avec historique et transactions fiables.

Auto Loader
Lit seulement les fichiers nouveaux d'un dossier.

Pipeline Lakeflow
Vous décrivez les tables, Databricks calcule l'ordre.

Streaming table
N'ajoute que les nouveautés à chaque passage.

Vue matérialisée
Une requête dont le résultat est stocké et rafraîchi.

AUTO CDC · SCD 2
Fusionner sur une clé ; garder chaque version.

Job
Un enchaînement de tâches planifié et surveillé.

PHASE 2 · MÉDAILLON

04

Le pipeline Lakeflow et la couche bronze

Transformer les fichiers en tables Delta, de façon incrémentale. On construit couche par couche et on exécute après chacune : une erreur se localise immédiatement.

20 min | Serverless · mode standard | Jobs & Pipelines

Déclaratif : vous décrivez les tables à obtenir, pas l'ordre des calculs. Databricks lit tous les fichiers du pipeline, déduit les dépendances et exécute dans le bon ordre.

Créer et régler le pipeline

1. **Jobs & Pipelines**, **Create**, **ETL pipeline**. Nom : `helios_pipeline`. Catalogue par défaut **helios**, schéma **bronze**.
2. Choisissez **Add existing assets** : dossier racine `02_pipeline`, chemin source `02_pipeline/transformations`. Si l'option n'apparaît pas, ouvrez **Settings**, rubrique **Source code**, et ajoutez ce même dossier.
3. Pour construire couche par couche, n'ajoutez d'abord que `01_bronze.py` (sélection de fichiers), puis les suivants aux étapes 5 et 6.
4. **Settings** : **Compute** = **Serverless** ; décochez **Performance optimized** (mode standard : démarrage de 4 à 6 minutes, coût réduit).
5. **Advanced settings**, **Configuration**, **Add configuration** : clé `spark.sql.session.timeZone`, valeur `UTC`. **Save**.
6. **Run pipeline**. Si une boîte du graphe devient rouge, le panneau **Issues** donne le message exact.

LE CODE · 02_PIPELINE/TRANSFORMATIONS/01_BRONZE.PY

Une seule fonction, `lire_enveloppes(famille)`, alimente deux tables ; une troisième lit le fichier texte du GFZ. L'essentiel tient en cinq lignes :

```
spark.readStream.format("cloudFiles")       # Auto Loader : mémorise les fichiers lus
  .option("cloudFiles.format", "json")
  .schema(ENVELOPPE)                        # schéma imposé : aucune surprise
  .load(f"{BRUT}/{famille}/*/*.json")
  .select("*", F.col("_metadata.file_path").alias("fichier"))  # traçabilité
```

`@dp.table` déclare la table et son commentaire ; `@dp.expect_or_fail` arrête tout si une enveloppe arrive vide.

VÉRIFIER

`03_verifications/verifications.sql` bloc « Étape 4 » : **swpc 5**, **donki ≈ 309**, **gfz_lignes ≈ 34 670** lignes.

Relancez **Run pipeline** sans rien changer : les compteurs **n'augmentent pas**. C'est la preuve de l'incrémental.

L'ŒIL DU GOUROU · STREAMING TABLE

Une streaming table n'ajoute que les nouveautés. Le point de reprise d'Auto Loader rend chaque passage quotidien minuscule : on ne paie que pour les quelques fichiers du jour, jamais pour l'historique déjà chargé.

PHASE 2 · MÉDAILLON

05

La couche silver : décoder, contrôler, dédoublonner, historiser

Trois fichiers, un par producteur de données. Ajoutez-les au pipeline un par un et exécutez après chacun.

30 min · 9 tables

TECHNIQUE	CE QU'ELLE FAIT	LE PIÈGE QU'ELLE RÉSOUT
`from_json` + `explode`	Le texte JSON du payload devient des colonnes, puis une liste de mesures devient une ligne par mesure	Un fichier contient des milliers de mesures
`@dp.temporary_view`	Étape intermédiaire, non stockée	Préparer les lignes avant de les fusionner
Expectations	`expect` signale · `expect_or_drop` écarte · `expect_or_fail` arrête	Mesurer et prouver la qualité (vitesse entre 200 et 3 000 km/s, Kp entre 0 et 9…)
AUTO CDC type 1	Une ligne par clé, la plus récente gagne	Chaque collecte NOAA couvre 24 h : sans cela, chaque minute serait en double
AUTO CDC type 2	Garde toutes les versions avec `__START_AT` et `__END_AT`	La NASA révise ses fiches CME : on conserve l'historique

FICHIER	TABLES PRODUITES	À RETENIR
`02_silver_noaa.py`	`vent_solaire`, `champ_magnetique`, `kp_noaa`, `rayons_x`, `cycle_solaire_mensuel`	Clé **(time_tag, sonde)** pour ne pas mélanger SOLAR1, ACE et IMAP ; `sequence_by = collecte_le` ; classe d'éruption A à X calculée depuis le flux
`03_silver_gfz.py`	`kp_historique_3h`	`split` découpe la ligne, `posexplode` déplie les 8 Kp du jour en 8 lignes, `timestampadd` convertit la tranche en heure ; les `-1` deviennent vides
`04_silver_donki.py`	`donki_cme` (SCD 2), `donki_gst`, `donki_flr`	Liens cause à effet conservés (`evenements_lies`) ; l'analyse « la plus précise » de chaque CME est extraite avec `filter`

LE CŒUR DU DÉDOUBLONNAGE · L'HISTORISATION DES CME

```
dp.create_auto_cdc_flow(
    target="helios.silver.donki_cme", source="v_cme", keys=["cme_id"],
    sequence_by=F.struct("version", "collecte_le"),  # version NASA, puis collecte
    stored_as_scd_type=2,                            # chaque révision = une ligne datée
    track_history_except_column_list=["collecte_le"])  # pas de fausse version
```

VÉRIFIER

Graphe : **3 bronze → 9 silver**. Bloc « Étape 5 » de `verifications.sql` :

- 3 sondes, environ 24 h chacune ;
- historique Kp depuis le **1932-01-01**, plus de 270 000 tranches ;
- tempête du **10 mai 2024**, Kp max 9.

L'ŒIL DU GOUROU · VOIR LA QUALITÉ

Cliquez une table du graphe (par exemple `vent_solaire`) : le panneau des expectations affiche, contrôle par contrôle, les lignes passées, signalées et écartées. La qualité n'est plus une promesse, c'est un chiffre.

PHASE 2 · MÉDAILLON

06

La couche gold : des mesures aux réponses

Sept vues matérialisées en SQL, recalculées par le pipeline à chaque passage. Une table gold, une question.

15 min · `02_pipeline/transformations/05_gold.sql`

Ajoutez `05_gold.sql` au pipeline, puis **Run pipeline**. Le graphe complet affiche **3 bronze, 9 silver, 7 gold**.

VUE GOLD	UNE LIGNE PAR	LA QUESTION À LAQUELLE ELLE RÉPOND
`vent_solaire_horaire`	heure	Le vent charge-t-il la magnétosphère ? Bz sud, pression, couplage de Newell
`comparaison_sondes`	heure et sonde	SOLAR1, ACE et IMAP voient-elles le même vent ? De combien s'écartent-elles ?
`kp_3h`	tranche de 3 h	Kp de 1932 à maintenant en une seule série : GFZ pour l'histoire, NOAA pour les dernières heures
`dim_cycles_solaires`	cycle	Dans quel cycle solaire (16 à 25) tombe chaque jour ?
`journal_geomagnetique`	jour depuis 1932	Kp max, classe NOAA G1 à G5, Ap, taches solaires, flux F10.7 et cycle du jour
`du_soleil_a_la_terre`	tempête et CME	Combien d'heures entre le départ d'une CME et la tempête, contre le trajet à vitesse constante ?
`eruptions_mensuelles`	mois et classe	Combien d'éruptions A, B, C, M, X par mois, et quelle est la plus forte ?

TROIS CALCULS DE PHYSICIEN, ÉCRITS EN SQL

INDICATEUR	FORMULE	CE QU'IL DIT
Couplage de Newell	$v^{4/3} \cdot B_T^{2/3} \cdot \sin^{8/3}(\theta/2)$	L'énergie que le vent injecte dans le bouclier terrestre : la **cause**, quand le Kp mesure la conséquence
Pression dynamique	$1{,}6726 \cdot 10^{-6} \cdot n \cdot v^2$ (nPa)	Un vent calme (5 /cm³, 400 km/s) pèse environ 1,3 nPa
Transit balistique	`149 597 870,7 km / v`	L'écart avec le trajet observé révèle si la CME a été freinée ou poussée

Classes de tempête : la NOAA raisonne par tiers de Kp ; 5- compte en G1 et 9- en G4, comme dans son échelle officielle. Seule la version courante de chaque CME est utilisée (`__END_AT IS NULL`).

VÉRIFIER LE FIL ROUGE

Bloc « Étape 6 » de `verifications.sql` : la tempête **2024-05-10T15:00:00-GST-001**, Kp max 9, reliée aux CME parties du Soleil les 8 et 9 mai. La seconde requête liste les dix journées les plus agitées depuis 1932.

L'ŒIL DU GOUROU · POURQUOI DES VUES MATÉRIALISÉES

Le résultat est stocké et recalculé de façon incrémentale quand c'est possible. Power BI lit des tables déjà prêtes : rapide à charger, et l'entrepôt SQL ne travaille presque pas.

PHASE 3 · EXPLOITER

07

Orchestrer : un job qui collecte, transforme et se contrôle

Ce qui distingue un projet de production d'un exercice : tout s'enchaîne seul chaque matin, et vous êtes prévenu si quelque chose cloche.

15 min · Jobs & Pipelines

Le notebook `01_notebooks/02_controle.py` lève une erreur si le vent solaire a plus de 6 h de retard, le Kp plus de 12 h, si l'historique compte moins de 34 000 jours ou s'il n'existe aucun lien CME vers tempête. Exécutez-le une fois à la main : il doit finir par « Tous les contrôles HÉLIOS sont au vert ».

1. **Jobs & Pipelines**, **Create**, **Job**. Nom : `helios_orchestration`.
2. Tâche `collecter` : type **Notebook**, source **Workspace**, chemin `01_notebooks/01_collecteur`, calcul **Serverless**. **Parameters** : clé `gfz`, valeur `auto`. **Retries** : 1.
3. **Add task**, **Pipeline** : `transformer`, pipeline **helios_pipeline**, dépend de collecter.
4. **Add task**, **Notebook** : `controler`, chemin `01_notebooks/02_controle`, dépend de transformer.
5. Panneau de droite : **Performance optimized** décoché ; **Job notifications**, **On failure**, votre e-mail.
6. **Schedules & Triggers**, **Scheduled**, **Advanced** (syntaxe cron) : `0 17 6 * * ?`, fuseau **Europe/Paris**.
7. **Run now** : les trois tâches passent au vert l'une après l'autre (10 à 15 minutes).

L'ŒIL DU GOUROU · 06:17 ET NON 06:00

Lecture du cron : seconde 0, minute 17, heure 6, tous les jours. À l'heure ronde, des milliers de robots interrogent les mêmes API publiques ; décaler de quelques minutes réduit les échecs. La nouvelle tentative unique absorbe la panne passagère d'un serveur sans masquer une vraie anomalie.

PHASE 3 · EXPLOITER

08

Gouverner et prouver : droits, tags, lignée, qualité, coût

Un projet se prouve : d'où vient chaque chiffre, qui peut le lire, quelle est sa qualité, combien il a coûté.

15 min · SQL Editor · Catalog

PREUVE	FICHIER OU GESTE	CE QUE VOUS OBTENEZ
Droits	`04_gouvernance/gouvernance.sql`	`account users` lit la couche gold uniquement : le moindre privilège
Tags	même fichier	Couche sur chaque schéma ; source et licence sur chaque table silver
Lignée	**Catalog**, helios, gold, `du_soleil_a_la_terre`, onglet **Lineage**, **See lineage graph**	Du volume brut à la table gold, construite par Databricks sans une ligne de code
Qualité	`04_gouvernance/qualite.sql`	`event_log` : contrôle par contrôle, lignes passées et en échec
Coût	`04_gouvernance/cout.sql`	`system.billing` : DBU et dollars par jour et par produit (SQL, DLT, JOBS…)

ORDRE DE GRANDEUR MESURÉ SUR CE PROJET

Construction complète, reprise historique comprise : **environ 17 USD** au prix catalogue Databricks, dont près de **60 % pour l'entrepôt SQL** (requêtes de vérification, tableau de bord). Le pipeline et le job, grâce à l'incrémental et au mode standard, pèsent peu. La facture réelle se lit dans le portail Azure, Gestion des coûts.

PHASE 4 · RESTITUER

09

Montrer dans Databricks : tableau de bord AI/BI et Genie

Avant Power BI, Databricks restitue lui-même : un tableau de bord pour l'œil, un agent pour les questions en français.

20 min · Dashboards · Genie

1. **Dashboards**, **Create dashboard**. Titre : `HÉLIOS · Météo spatiale`.
2. Onglet **Data**, **Create from SQL** : créez les trois jeux de `05_restitution/tableau_de_bord.sql`.
3. Onglet de la page : **Add a visualization** pour chacun des quatre visuels ci-dessous, puis **Publish**.

VISUEL	JEU	TYPE
Jours G4 et G5 par année	1	Barres
Taches solaires depuis 1932	1	Ligne
Vitesse d'une CME et temps de trajet	2	Nuage de points
Bz et vitesse sur 7 jours	3	Ligne

1. **Genie**, **New**. Ajoutez les quatre tables gold listées dans `05_restitution/genie.md` et votre entrepôt.
2. Titre, instructions générales et trois questions d'exemple : copiez-les depuis ce même fichier.
3. Posez « Quelles sont les 10 journées les plus agitées depuis 1932 ? », puis cliquez la réponse pour lire le SQL généré.

L'ŒIL DU GOUROU

Les instructions de Genie fixent le vocabulaire métier (Kp de 0 à 9, « tempête forte » = G4 ou G5, heures en UTC) : c'est ce qui transforme un générateur de SQL en analyste fiable.

PHASE 4 · RESTITUER

10

Passer le relais à Power BI et Fabric Apps

Le lakehouse est prêt. Dernier geste côté Databricks : brancher Power BI Desktop sur la couche gold, la porte d'entrée de Fabric.

10 min · Power BI Desktop

1. Dans Databricks : **SQL Warehouses**, votre entrepôt, onglet **Connection details**. Copiez **Server hostname** et **HTTP path**.
2. Power BI Desktop : **Accueil**, **Obtenir les données**, **Plus**, cherchez `Azure Databricks`, **Se connecter**.
3. Collez le nom d'hôte et le chemin HTTP. Mode : **Import**. Authentification : **Compte d'organisation**, puis **Se connecter**.
4. Dans le navigateur, dépliez **helios**, **gold** : cochez les **sept** vues. Dans **silver**, cochez aussi `cycle_solaire_mensuel` (taches depuis 1749). **Charger**.

VÉRIFIER

Les huit tables apparaissent dans le volet **Données**. `journal_geomagnetique` compte plus de **34 000 lignes**, une par jour depuis 1932.

L'ŒIL DU GOUROU · IMPORT PLUTÔT QUE DIRECTQUERY

En Import, l'entrepôt ne se réveille qu'au chargement et aux actualisations planifiées, jamais pendant la navigation dans le rapport. Le gold est déjà agrégé : le modèle reste léger et rapide.

LA SUITE, CÔTÉ FABRIC

Modèle sémantique tables, relations, mesures DAX → **Rapport Power BI** publié dans un espace Fabric → **Fabric Apps** application React qui lit le modèle en DAX

EXPLOITATION

Couper le coût, dépanner, vérifier

Un projet terminé ne doit plus rien consommer quand personne ne s'en sert. Les données restent ; seul le calcul s'arrête.

! METTRE HÉLIOS EN SOMMEIL, SANS RIEN SUPPRIMER

1. **Jobs & Pipelines**, helios_orchestration, **Schedules & Triggers**, **Pause**.
2. **SQL Warehouses** : **Stop** si l'entrepôt tourne ; vérifiez **Auto stop**.
3. **Compute** : aucune ligne **Running**, sinon **Terminate**.
4. Deux jours plus tard : portail Azure, **Analyse des coûts**, **Grouper par : Ressource**. Plus rien ne doit augmenter, à part quelques centimes de stockage.

Pour tout effacer un jour : supprimer le job, puis le pipeline, puis `DROP CATALOG helios CASCADE`.

DÉPANNAGE

SYMPTÔME	CAUSE ET REMÈDE
`ConnectionError`	Le serverless ne sort pas sur Internet : politique réseau du compte
`PERMISSION_DENIED`	Objet absent ou créé par un autre : revoir l'étape 1
« table not found »	Le fichier qui la produit n'est pas encore dans le pipeline : ajouter dans l'ordre 01 à 05
Boîte rouge dans le graphe	Panneau **Issues** : le message désigne le fichier et la ligne
Aucune ligne nouvelle	Normal si aucun fichier n'est arrivé : relancer d'abord le collecteur
Démarrage de 5 minutes	Mode standard, moins cher : attendre

LISTE DE CONTRÔLE FINALE

- ☐ **00** Espace Premium, entrepôt 2X-Small à arrêt automatique, Git folder importé
- ☐ **01** Catalogue helios, 4 schémas, volume brut, commentaires et tags
- ☐ **02** Les quatre cellules d'exploration affichent les données
- ☐ **03** Reprise historique : ~315 fichiers ; widgets remis en quotidien
- ☐ **04** 3 tables bronze ; une relance n'ajoute rien
- ☐ **05** 9 tables silver ; historique Kp depuis 1932
- ☐ **06** 7 vues gold ; tempête du 10 mai 2024 reliée à ses CME
- ☐ **07** Job au vert, planifié à 06:17, e-mail en cas d'échec
- ☐ **08** Droits gold, tags, capture de la lignée, coût relevé
- ☐ **09** Tableau de bord publié ; Genie répond en français
- ☐ **10** Huit tables chargées dans Power BI en mode Import
- ☐ Job mis en **Pause** après la démonstration

SOURCES, LICENCES ET RÉFÉRENCES

NOAA SWPC : vent solaire, Kp, rayons X, cycle solaire (domaine public) · **NASA CCMC DONKI** : CME, tempêtes, éruptions et leurs liens (domaine public) · **GFZ Potsdam** : Kp, ap, Ap depuis 1932 (CC BY 4.0). Références : documentation Databricks (Lakeflow, Auto Loader, AUTO CDC, Unity Catalog, tables système) ; Newell et al. (2007), couplage vent solaire et magnétosphère.

HÉLIOS · Abdoul Hamid Diallo · github.com/abdoulhamiddiallo/helios-databricks